

• VERIQTA •

- ✓ High Performance
- ✓ Secure
- ✓ Reliable
- ✓ Scalable
- ✓ Production Ready

Master  
MySQL for  
Real-World  
Production  
Systems.

# MySQL

## PRODUCTION HANDBOOK

From Fundamentals to Enterprise  
Database Operations

### WHAT YOU WILL MASTER

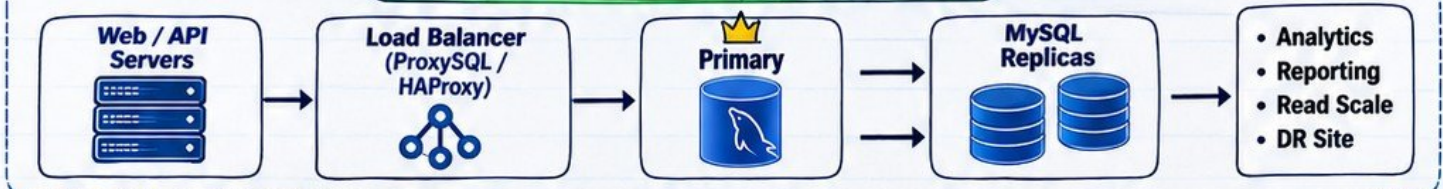
- ✓ MySQL Architecture
- ✓ Query Optimization
- ✓ Indexing Strategies
- ✓ Transactions & ACID
- ✓ Replication & HA
- ✓ Backup & Recovery
- ✓ Performance Tuning
- ✓ Security & Compliance
- ✓ Advanced Operations
- ✓ Production Troubleshooting



### KEY FEATURES

- ⚡ Production Best Practices
- 📊 Performance Optimization
- 🛡️ Security & High Availability
- 💾 Backup & Disaster Recovery
- ⚙️ Real-World Case Studies

### MySQL Production Architecture



### WHO IS THIS HANDBOOK FOR?

- ✓ Database Administrators (DBAs)
- ✓ DevOps & Cloud Engineers
- ✓ Platform Engineers
- ✓ Backend Developers
- ✓ Site Reliability Engineers
- ✓ Anyone building production systems

Build, Operate and Scale  
Reliable Database Systems  
*Like a Pro.*

Great applications  
need great databases.  
*Master MySQL.*

VERIQTA

ENGINEER THE FUTURE. AUTOMATE EVERYTHING.



YouTube  
/veriqta



GitHub  
/veriqta



LinkedIn  
/veriqta



X (Twitter)  
@veriqta



Instagram  
@veriqta



Telegram  
@veriqta

# INTRODUCTION TO MySQL PRODUCTION SYSTEMS

FROM FUNDAMENTALS TO ENTERPRISE DATABASE OPERATIONS



**GREAT DATABASES DON'T HAPPEN BY ACCIDENT.**

- ✓ Design Smart
- ✓ Operate Reliably
- ✓ Optimize Performance
- ✓ Ensure Availability
- ✓ Deliver Value

## ★ WHY MySQL POWERS ENTERPRISE APPLICATIONS



**Proven Reliability**  
Battle-tested by millions of businesses worldwide.



**High Performance**  
Optimized for speed, concurrency and low latency.



**Cost Effective**  
Open source with enterprise-grade capabilities.



**Scalable & Flexible**  
Handles small apps to massive enterprise workloads.



**Secure & Compliant**  
Strong security, roles, encryption and audit features.



**Rich Ecosystem**  
Tools, connectors, monitoring and strong community support.

## ★ OLTP vs OLAP WORKLOADS

### OLTP

Online Transaction Processing



- Many short transactions
- High concurrency (many users)
- INSERT, UPDATE, DELETE, SELECT
- ACID compliance critical
- Row-oriented operations
- Example: E-commerce, Banking, ERP

VS

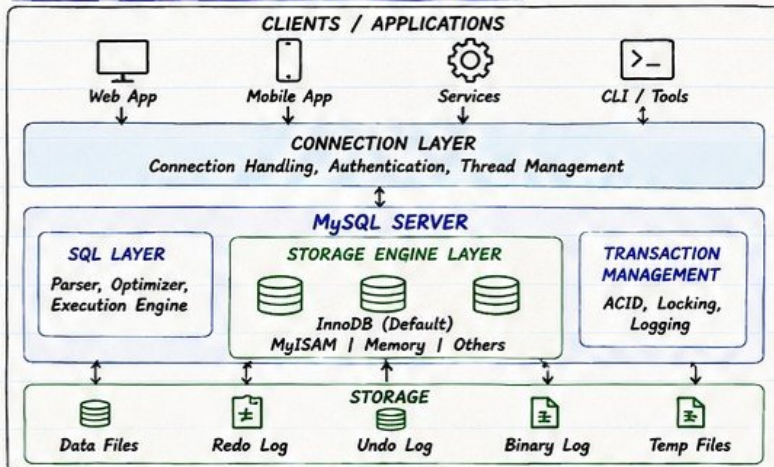
### OLAP

Online Analytical Processing



- Complex analytical queries
- Large data volumes
- Aggregations & reporting
- Read-heavy workloads
- Column-oriented operations
- Example: Data Warehouse, Reporting, BI

## ★ MySQL ARCHITECTURE OVERVIEW



## ★ PRODUCTION DATABASE LIFECYCLE



**1. PLAN & DESIGN**  
Understand requirements, model data and design schema.



**2. BUILD & CONFIGURE**  
Install, configure and harden MySQL for production.



**3. DEPLOY & OPERATE**  
Deploy to production and ensure high availability.



**4. MONITOR & OPTIMIZE**  
Monitor performance, tune queries and optimize resources.



**5. BACKUP & PROTECT**  
Backup data regularly and test recovery procedures.



**6. SCALE & EVOLVE**  
Scale as needed and evolve with business growth.

## ★ COMMON ENTERPRISE USE CASES



**E-COMMERCE**  
Orders, inventory, customers, payments



**BANKING & FINANCE**  
Transactions, accounts, loans, compliance



**ERP SYSTEMS**  
HR, payroll, finance, supply chain



**SAAS APPLICATIONS**  
Multi-tenant data, user management



**ANALYTICS & BI**  
Reporting, dashboards, business intelligence



**IOT & TELEMETRY**  
Device data, events, monitoring

“A reliable database is the foundation of a reliable system.”  
— VERIQTA



### PRODUCTION TIP

Start with a solid design, monitor everything, and automate your operations.

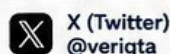


### REMEMBER

Great applications rely on great data.  
Great data relies on great databases.  
Great databases rely on great engineers.



MASTER MySQL. POWER ENTERPRISE. DELIVER VALUE.



# INSTALLING AND SECURING



# MySQL



## BUILD A SECURE FOUNDATION FOR YOUR PRODUCTION DATABASE

### 1 INSTALLATION METHODS



**Linux Package Manager**  
apt, yum, dnf, zypper



**Docker**  
Official MySQL Docker image



**MySQL Community Installer**  
Windows / Linux / macOS



**Cloud Managed Services**  
Amazon RDS, Google Cloud SQL,  
Azure Database for MySQL



**Source Compilation**  
From official MySQL source code

### 2 DIRECTORY STRUCTURE

```
/etc/mysql/  
├── # Configuration files  
├── /var/lib/mysql/  
│   └── # Data directory  
├── /var/log/mysql/  
│   └── # Log files  
├── /var/run/mysql/  
│   └── # Socket file  
├── /usr/bin/  
│   └── # Binaries  
├── /usr/sbin/  
│   └── # Server binaries  
└── /usr/share/mysql/  
    └── # Support files
```

### 3 INITIAL CONFIGURATION



Edit my.cnf / my.ini

Key settings:

- ✓ character-set-server = utf8mb4
- ✓ collation-server = utf8mb4\_unicode\_ci
- ✓ innodb\_buffer\_pool\_size
- ✓ max\_connections
- ✓ log\_error = /var/log/mysql/error.log
- ✓ general\_log = OFF (disable in prod)
- ✓ skip\_name\_resolve = ON
- ✓ secure\_file\_priv = /var/lib/mysql-files

### 4 ROOT ACCOUNT SECURITY



- ✓ Never leave root accessible remotely.
- ✓ Use strong passwords.
- ✓ Restrict root login to localhost.
- ✓ Remove anonymous users.
- ✓ Remove test database.
- ✓ Use a dedicated admin account if needed.

#### SECURE ROOT ACCOUNT EXAMPLE

```
-- Login to MySQL  
mysql -u root -p  
  
-- Set strong root password  
ALTER USER 'root'@'localhost'  
IDENTIFIED BY 'StrongP@ssw0rd!';  
  
-- Remove remote root login  
DELETE FROM mysql.user WHERE User='root'  
AND Host!='localhost';  
  
-- Remove anonymous users  
DELETE FROM mysql.user WHERE User='';  
  
-- Remove test database  
DROP DATABASE IF EXISTS test;  
  
FLUSH PRIVILEGES;
```

### 5 AUTHENTICATION PLUGINS

| PLUGIN                | DESCRIPTION                              | USE CASE                   |
|-----------------------|------------------------------------------|----------------------------|
| mysql_native_password | Legacy authentication                    | Compatibility              |
| caching_sha2_password | Default in MySQL 8.0+<br>(secure & fast) | Recommended                |
| sha256_password       | Secure<br>(RSA based)                    | Older clients<br>with SSL  |
| auth_socket           | Use OS user<br>(Linux)                   | Local admin<br>access      |
| pam                   | Use PAM                                  | Enterprise<br>environments |



Use **caching\_sha2\_password** for new deployments.  
Avoid **mysql\_native\_password** unless required  
for legacy applications.

### 6 SECURITY HARDENING CHECKLIST

- |                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>✓ Keep MySQL up to date.</li><li>✓ Disable unnecessary services and ports.</li><li>✓ Bind MySQL to localhost or trusted IPs.</li><li>✓ Use SSL/TLS for connections.</li><li>✓ Enforce strong password policy.</li><li>✓ Limit privileges (least privilege principle).</li><li>✓ Disable unnecessary databases and features.</li></ul> | <ul style="list-style-type: none"><li><input type="checkbox"/> Enable the binary log.</li><li><input type="checkbox"/> Enable the error log.</li><li><input type="checkbox"/> Enable slow query log.</li><li><input type="checkbox"/> Set secure_file_priv.</li><li><input type="checkbox"/> Disable symbolic links.</li><li><input type="checkbox"/> Configure firewall rules.</li><li><input type="checkbox"/> Regular backups and test restores.</li></ul> |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### EXAMPLE: BIND TO LOCALHOST ONLY

```
# my.cnf / my.ini  
[mysqld]  
bind-address = 127.0.0.1  
# or specific IP  
# bind-address = 10.0.0.5
```



“ Security is not  
a feature.  
It is the  
foundation of  
trust in your  
database.  
– VERIQTA ”

### COMMAND QUICK REFERENCE



Check MySQL Status  
systemctl status mysql  
  
Start MySQL  
systemctl start mysql

Enable MySQL  
systemctl enable mysql  
  
Restart MySQL  
systemctl restart mysql

Secure Installation  
mysql\_secure\_installation  
  
Login as Root  
mysql -u root -p

Check Version  
mysql --version  
  
Show Variables  
SHOW VARIABLES;

#### PRODUCTION TIP

A secure database  
is a reliable database.  
Secure today,  
sleep well tomorrow.  
– VERIQTA



SECURE YOUR DATABASE. PROTECT YOUR BUSINESS.

# MySQL ARCHITECTURE DEEP DIVE

Understand how MySQL processes every request from the client to the storage engine.

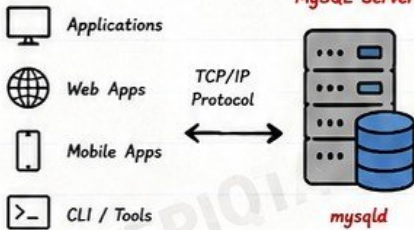
## KEY TAKEAWAY

MySQL is a multi-threaded client-server database. Every connection is handled efficiently from networking to storage.

— VERIQTA

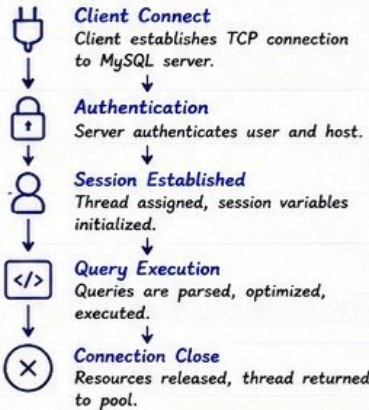
## 1 CLIENT-SERVER ARCHITECTURE

### Clients



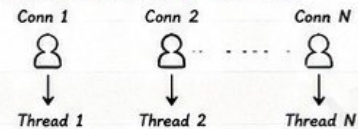
Clients connect over TCP/IP. Server listens on port 3306 (default).

## 2 CONNECTION LIFECYCLE

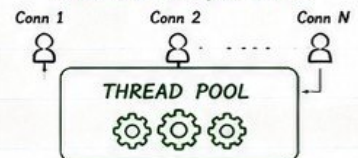


## 3 THREADS AND THREAD POOLS

### Classic (One Thread Per Connection)



### Thread Pool (Enterprise / 8.0+)



- ✓ Improves scalability and reduces context switching.
- ✓ Reuses threads for better performance.

## 4 STORAGE ENGINE LAYER

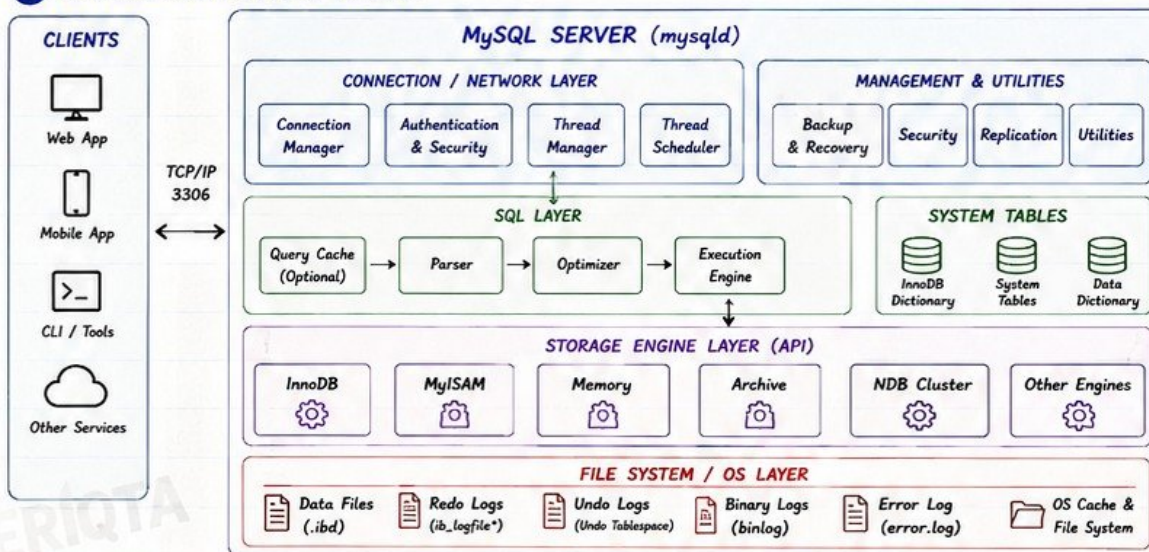
MySQL supports multiple storage engines. Default: InnoDB

- ✓ InnoDB - ACID compliant, row-level locking
- ✓ MyISAM - Fast reads, table-level locking
- ✓ Memory - In-memory tables
- ✓ Archive - Compressed, read mostly
- ✓ NDB Cluster - Distributed storage
- ✓ Federated - Access remote data
- ✓ Custom Engines - Extensible

## 5 QUERY EXECUTION FLOW



## 6 INTERNAL ARCHITECTURE DIAGRAM



### ARCHITECTURE HIGHLIGHTS

- ✓ Multi-threaded client-server model.
- ✓ Pluggable storage engine architecture.
- ✓ SQL layer handles parsing, optimization and execution.
- ✓ Storage engines handle actual data storage and retrieval.
- ✓ Logs ensure durability, recovery and replication.



“Great performance comes from understanding how MySQL handles connections, queries, and data behind the scenes.”  
— VERIQTA



### PRO TIP

Monitor threads, connections, and query performance continuously. A slow query today can be an outage tomorrow.

# DATABASE DESIGN BEST PRACTICES

Design databases that are reliable, scalable,  
maintainable, and ready for production.

## KEY TAKEAWAY

Good design today  
prevents problems  
tomorrow and supports  
scale, performance,  
and data integrity.

— VERIQTA

## 1 NORMALIZATION

Organize data to minimize redundancy  
and dependency.

- ✓ 1NF: Atomic values, no repeating groups
- ✓ 2NF: No partial dependency on part of a key
- ✓ 3NF: No transitive dependency
- ✓ Reduces duplication and update anomalies
- ✓ Improves data integrity

### UNNORMALIZED

#### Orders (UNF)

| OrderID | CustomerName | ProductName | ProductPrice |
|---------|--------------|-------------|--------------|
| 1       | Alice        | Laptop      | 1200         |
| 1       | Alice        | Mouse       | 25           |
| 2       | Bob          | Keyboard    | 45           |

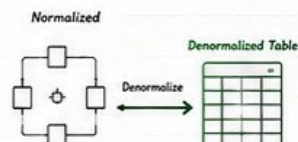
### NORMALIZED (3NF)

| Customers       | Orders          | OrderItems                        |
|-----------------|-----------------|-----------------------------------|
| CustomerID (PK) | OrderID (PK)    | OrderID (FK) ProductID (FK) Price |
| CustomerName    | CustomerID (FK) |                                   |
| 1 Alice         | 1 1             | 1 101 1200                        |
| 2 Bob           | 2 2             | 2 102 25                          |
|                 |                 | 2 103 45                          |

## 2 DENORMALIZATION

Intentionally add redundancy to improve  
read performance.

- ✓ Use for reporting, dashboards, and read-heavy workloads
- ✓ Pre-compute summaries
- ✓ Materialized views
- ✓ Cache tables
- ✓ Always balance performance vs consistency



Denormalize only when  
measured performance  
gains are significant.

## 3 PRIMARY KEYS



Uniquely identify each row

- ✓ Use surrogate keys (INT/BIGINT AUTO\_INCREMENT or UUID)
- ✓ Avoid relying on natural keys
- ✓ Keep keys short and immutable
- ✓ Always define PRIMARY KEY

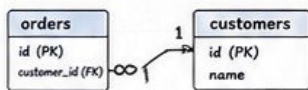
| GOOD                                       | BAD                              |
|--------------------------------------------|----------------------------------|
| id BIGINT<br>AUTO_INCREMENT<br>PRIMARY KEY | name VARCHAR(255)<br>PRIMARY KEY |

## 4 FOREIGN KEYS



Enforce relationships  
between tables

- ✓ Maintain referential integrity
- ✓ Use ON UPDATE / DELETE rules wisely
- ✓ Index all foreign keys
- ✓ Avoid circular dependencies



## 5 DATA TYPES



Choose the right type  
for the job

- ✓ Use the smallest type that fits
- ✓ Avoid TEXT/BLOB unless necessary
- ✓ Use UNSIGNED for non-negative numbers
- ✓ Use DECIMAL for money, not FLOAT/DOUBLE

### EXAMPLES

|                    |                      |
|--------------------|----------------------|
| INT / BIGINT       | IDs, counts          |
| VARCHAR(n)         | Short text           |
| TEXT               | Large text           |
| DATE / DATETIME    | Dates & time         |
| DECIMAL(p,s)       | Money, precise       |
| BOOLEAN/TINYINT(1) | Flags                |
| JSON               | Semi-structured data |

## 6 NAMING CONVENTIONS

- ✓ Use consistent, descriptive names
- ✓ Use lowercase letters
- ✓ Use underscores, not spaces
- ✓ Use singular table names
- ✓ Avoid reserved keywords
- ✓ Be consistent across the schema

### GOOD EXAMPLES

customer  
order  
order\_item  
product\_category  
created\_at  
updated\_at

### BAD EXAMPLES

CustomerTbl  
Order Items  
PRODUCTCATEGORY  
createAt

## 7 PRODUCTION SCHEMA DESIGN

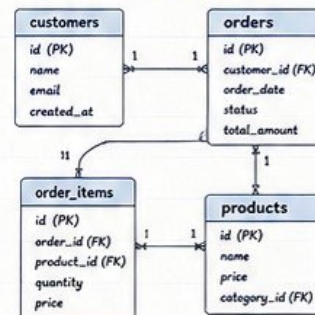
### DESIGN PRINCIPLES

- ✓ **Simplicity** - First Keep it simple and flexible.
- ✓ **Integrity** - Enforce rules at the database level.
- ✓ **Performance** - Index, partition, and optimize for access patterns.
- ✓ **Scalability** - Design for growth and high availability.
- ✓ **Maintainability** - Easy to understand, document, and evolve.

### PRODUCTION SCHEMA CHECKLIST

- ✓ Clearly defined primary keys
- ✓ All foreign keys indexed
- ✓ Appropriate data types selected
- ✓ Default values where applicable
- ✓ Constraints and validations in place
- ✓ Indexes for query performance
- ✓ Audit columns (created\_at, updated\_at)
- ✓ Soft delete strategy (if needed)
- ✓ Partitioning strategy (for large tables)
- ✓ Documentation and ER diagram

### SAMPLE SCHEMA OVERVIEW



“A well-designed database  
is the foundation of  
a scalable application.”  
— VERIQTA



### PRO TIP

Design for change. Requirements will evolve.  
Build a schema that is flexible, observable,  
and easy to extend.



### REMEMBER

Good design reduces bugs, improves performance,  
and saves cost at scale.

# INDEXES FOR PERFORMANCE

THE RIGHT INDEX MAKES QUERIES FAST.  
THE WRONG INDEX MAKES PERFORMANCE WORSE.

## KEY TAKEAWAY

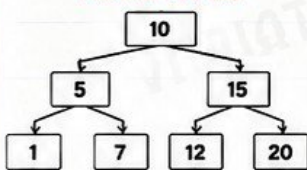
- ✓ Index what you search.
- ✓ Index what you join.
- ✓ Index what you sort.
- ✓ Do not over-index.
- ✓ Always measure.

— VERIQTA

## 1 CLUSTERED INDEXES

- In InnoDB, the PRIMARY KEY is the clustered index.
- Data is stored in the order of the primary key.

PRIMARY KEY (id)

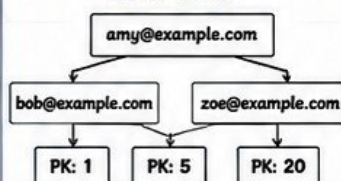


Only one clustered index per InnoDB table.

## 2 SECONDARY INDEXES

- Indexes created on columns other than the primary key.
- Contains the indexed column and the primary key value.

INDEX (email)



Points to the actual row using the primary key.

## 3 COMPOSITE INDEXES

- Indexes on multiple columns.
- Column order matters.
- Best for filtering, sorting and joins.

INDEX (last\_name, first\_name)

| last_name | first_name | PK |
|-----------|------------|----|
| Adams     | John       | 1  |
| Adams     | Mike       | 5  |
| Baker     | Alice      | 9  |
| Brown     | Carol      | 20 |



Most selective column should come first.

## 4 COVERING INDEXES

- Index contains all columns needed for the query.
- Avoids reading the actual table rows.

INDEX (status, created\_at, total)

```
SELECT status, created_at, total
FROM orders
WHERE status = 'paid'
ORDER BY created_at;
```



Query is satisfied entirely from the index.

## 5 FULL-TEXT INDEXES

- Designed for fast text searching.
- Works on TEXT, VARCHAR, or CHAR columns.
- Supports natural language queries.
- Uses inverted index under the hood.

Example

| id | content                        |
|----|--------------------------------|
| 1  | MySQL performance tuning guide |
| 2  | Advanced database indexing     |
| 3  | Full-text search in MySQL      |

```
CREATE FULLTEXT INDEX idx_content
ON articles(content);
```

## 6 COMMON INDEXING MISTAKES

- ✗ Indexing low-cardinality columns (e.g. gender, status with few values) → Low selectivity. Little performance gain.
- ✗ Too many indexes on a table → Slows down INSERT, UPDATE, DELETE. Uses more disk space.
- ✗ Indexing columns not used in WHERE, JOIN, or ORDER BY → Wastes storage and maintenance overhead.
- ✗ Redundant indexes (e.g., index on (a, b) and (a)) → The longer index already covers the prefix.
- ✗ Ignoring column order in composite indexes → Wrong order leads to index not being used.
- ✗ Not reviewing unused indexes → Unused indexes still cost performance.

## 7 QUERY OPTIMIZATION EXAMPLES

Example 1: Without Index

```
EXPLAIN SELECT * FROM orders
WHERE customer_id = 101;
```

| type | possible_keys | key  | rows   | Extra       |
|------|---------------|------|--------|-------------|
| ALL  | NULL          | NULL | 985432 | Using where |



Full table scan. Very slow on large tables.

Example 2: With Index

```
EXPLAIN SELECT * FROM orders
WHERE customer_id = 101;
```

| type | possible_keys   | key             | rows | Extra                 |
|------|-----------------|-----------------|------|-----------------------|
| ref  | idx_customer_id | idx_customer_id | 32   | Using index condition |



Index lookup. Fast and efficient.

## 8 INDEXING BEST PRACTICES

- ✓ Index columns used in WHERE, JOIN, GROUP BY, ORDER BY.
- ✓ Use the most selective column first in composite indexes.
- ✓ Use covering indexes for read-heavy queries.
- ✓ Review and remove unused indexes regularly.
- ✓ Test with EXPLAIN before and after adding indexes.
- ✓ Balance read performance with write performance.
- ✓ Monitor index size and fragmentation.

## CHECK BEFORE ADDING AN INDEX

- ✓ Is the column used in searches or joins?
- ✓ Will it reduce the number of rows scanned?
- ✓ Is the column selective enough?
- ✓ Is the index size reasonable?
- ✓ Will the write performance be impacted?



## PRO TIP

Measure. Don't guess.  
Use slow query log, EXPLAIN, and performance schema to validate.

— VERIQTA



“ Good indexes make reads fast.  
Great indexes make your  
entire system scalable. ”  
— VERIQTA



## REMEMBER

Indexes are not free.  
They speed up reads,  
but slow down writes.

## SQL QUICK CHECK

```
SHOW INDEX FROM orders\G
SHOW CREATE TABLE orders\G
EXPLAIN SELECT * FROM orders
WHERE status = 'paid';
```

## DOCUMENT YOUR INDEXES

- ✓ Purpose
- ✓ Columns
- ✓ Created by
- ✓ When
- ✓ Review date

# SQL QUERY OPTIMIZATION

WRITE BETTER QUERIES. GET FASTER RESULTS.

## KEY TAKEAWAY

- ✓ Understand before you optimize.
- ✓ Measure before you change.
- ✓ Index smart.
- ✓ Write for the optimizer.

— VERIQTA

## 1 EXPLAIN PLANS

- EXPLAIN shows how MySQL will execute your query.
- It reveals the access type, indexes used, rows examined, and extra operations.
- Use it for every slow query.

Example:

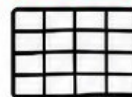
```
EXPLAIN SELECT * FROM orders
WHERE customer_id = 101;
```

| id | select_type | table  | type | key      | rows | Extra       |
|----|-------------|--------|------|----------|------|-------------|
| 1  | SIMPLE      | orders | ref  | idx_cust | 10   | Using index |

## 2 EXECUTION PLANS

- MySQL chooses an execution plan based on statistics.
- The plan shows the order of operations.
- Good plans use indexes and fewer rows.
- Bad plans cause full table scans and high cost.

Bad Plan  
(Full Scan)



orders

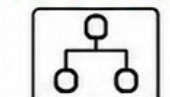
SCAN

Filter



Filter

Good Plan  
(Index Lookup)



idx\_customer\_id

Check rows examined. Lower is better.

## 3 JOIN OPTIMIZATION

- Join smaller tables first.
- Ensure join columns are indexed.
- Avoid functions on join columns.
- Use INNER JOIN when possible.
- Check the join order in EXPLAIN.

```
SELECT o.id, c.name
FROM orders o
JOIN customers c
ON o.customer_id = c.id
WHERE c.status = 'active';
```



## 4 SORTING

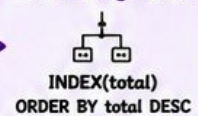
- Sorting is expensive.
- Use indexes to avoid filesort.
- Match ORDER BY with index order.
- Avoid sorting large result sets.
- Use LIMIT when possible.

Bad (Filesort)



ORDER BY  
o.total DESC

Good (Using Index)



INDEX(total)  
ORDER BY total DESC

## 5 FILTERING

- Filter early, filter smart.
- Use indexed columns in WHERE.
- Avoid functions on columns.
- Avoid leading wildcards.
- Use the most selective conditions.

Bad

```
WHERE
YEAR(order_date)
= 2024
```

Good

```
WHERE
order_date >= '2024-01-01'
AND order_date < '2025-01-01'
```

## 6 PAGINATION

- OFFSET gets slower as the number grows.
- Use keyset pagination for better performance.
- Always use a stable ORDER BY column.
- Avoid large OFFSET with LIMIT.

Bad (OFFSET)

```
SELECT * FROM orders
ORDER BY id
LIMIT 100000, 20;
```

Good (Keyset Pagination)

```
SELECT * FROM orders
WHERE id > 100000
ORDER BY id
LIMIT 20;
```

## 7 QUERY ANTI-PATTERNS

- ❌ SELECT \* → Select only needed columns.
- ❌ Using functions on indexed columns → Avoid functions; use ranges.
- ❌ Leading wildcard in LIKE '%term' → Use full-text index instead.
- ❌ OR conditions across columns → Use UNION or redesign.
- ❌ No WHERE clause on large tables → Always filter early.
- ❌ NOT IN (SELECT ...) with NULLs → Use NOT EXISTS instead.
- ❌ Large OFFSET for pagination → Use keyset pagination.

## QUICK CHECKLIST FOR BETTER QUERIES

- ✓ Use EXPLAIN for every slow query.
- ✓ Ensure proper indexes exist.
- ✓ Filter early and use selective conditions.
- ✓ Avoid functions on indexed columns.
- ✓ Optimize JOIN order and types.
- ✓ Avoid unnecessary sorting.
- ✓ Use LIMIT and keyset pagination.
- ✓ Monitor and update statistics.
- ✓ Avoid SELECT \*.
- ✓ Review slow query log.
- ✓ Test changes and measure.



PRO TIP:

Good queries are not just about speed — they are about scalability, consistency, and cost. The optimizer is smart, but your SQL should guide it.

# INNODB STORAGE ENGINE

THE HEART OF MYSQL TRANSACTIONS AND RELIABILITY.

## KEY TAKEAWAY

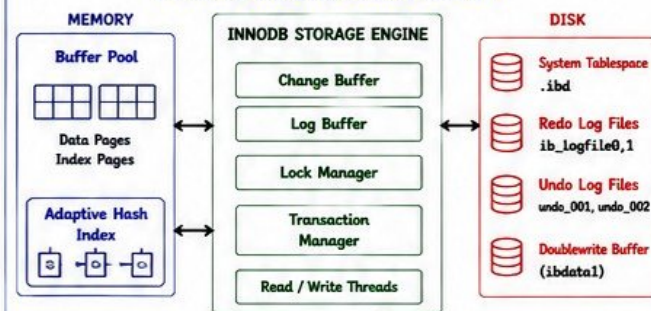
- ✓ Understand how InnoDB stores and protects data.
- ✓ Logs + Buffer Pool = Performance.
- ✓ InnoDB = ACID by design.

— VERIQTA

## 1 INNODB ARCHITECTURE

- InnoDB is the default transactional storage engine.
- It provides ACID compliance, row-level locking, crash recovery, and high performance.

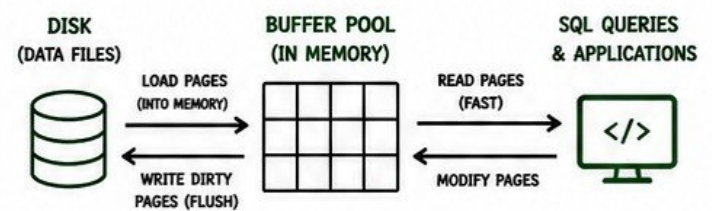
### INNODB ARCHITECTURE OVERVIEW



InnoDB coordinates memory, threads, and disk structures to deliver durable and consistent data.

## 2 BUFFER POOL

- In-memory cache for data and index pages.
- Reduces disk I/O and improves performance.
- Dirty pages are written to disk by background threads.
- Size is controlled by `innodb_buffer_pool_size`.

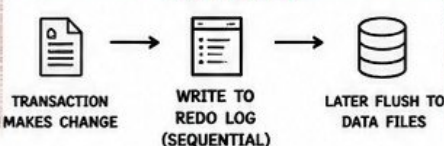


A larger buffer pool = fewer disk reads = better performance.

## 3 REDO LOG

- Records all changes before they are written to data files.
- Ensures durability (D in ACID).
- Stored in `ib_logfile0` and `ib_logfile1`.
- Written sequentially for high performance.

### HOW IT WORKS

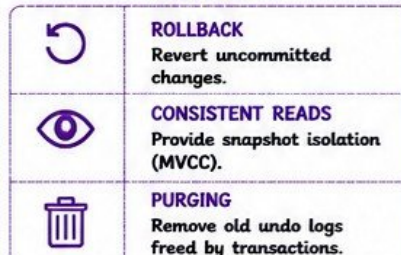


If MySQL crashes, InnoDB uses the redo log to recover committed transactions.

## 4 UNDO LOG

- Stores before-images of changed rows.
- Used for rollback and consistent reads.
- Each transaction has its own undo log.
- Older undo logs are purged automatically.

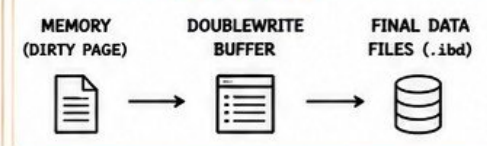
### USED FOR



## 5 DOUBLEWRITE BUFFER

- Protects against partial page writes.
- Pages are written to a special area first.
- Then written to their final location.
- Prevents corruption if MySQL crashes during write.

### WRITE PROCESS



Ensures pages are either fully written or not written at all.

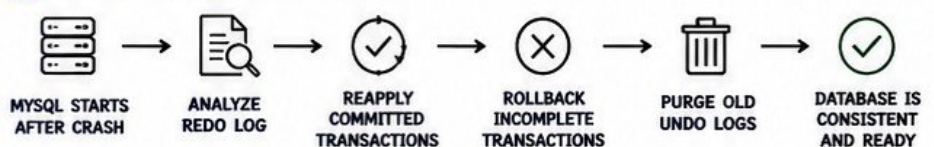
## 6 ADAPTIVE HASH INDEX (AHI)

- Automatically creates hash indexes on frequently searched data.
- Speeds up point lookups.
- Stored in memory inside the buffer pool.
- Enabled by default (`innodb_adaptive_hash_index`).



AHI improves performance without changes to your schema.

## 7 CRASH RECOVERY PROCESS



InnoDB ensures zero data loss for committed transactions and full consistency after recovery.

### INNODB RELIABILITY CHECKLIST

- |                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                             |                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>✓ Use InnoDB for all transactional workloads.</li> <li>✓ Size the buffer pool correctly (50–80% RAM).</li> <li>✓ Monitor redo log size and usage.</li> <li>✓ Keep undo logs from growing excessively.</li> </ul> | <ul style="list-style-type: none"> <li>✓ Ensure doublewrite buffer is enabled.</li> <li>✓ Monitor checkpoint age and I/O.</li> <li>✓ Regularly backup using logical or physical backups.</li> <li>✓ Test crash recovery in staging environments.</li> </ul> | <ul style="list-style-type: none"> <li>✓ Monitor slow queries and I/O waits.</li> <li>✓ Update MySQL to the latest stable version.</li> <li>✓ Review and tune InnoDB settings regularly.</li> </ul> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



### PRO TIP:

InnoDB's logs and buffers work together to give you speed, durability, and reliability. Understand them deeply — and your database will stay fast and safe.

# TRANSACTIONS AND ACID

ENSURING RELIABLE, CONSISTENT, AND SAFE DATABASE OPERATIONS.

## KEY TAKEAWAY

- ✓ ACID guarantees reliable transactions.
  - ✓ Transactions protect data from errors and failures.
  - ✓ Use transactions for critical operations.
- VERIQTA

### 1 ATOMICITY



All operations in a transaction succeed together or fail together.

#### EXAMPLE

Money transfer: Debit from one account and credit to another must both succeed or both fail.

### 2 CONSISTENCY



Transactions bring the database from one valid state to another valid state.

#### EXAMPLE

All constraints, rules, and data integrity are maintained before and after the transaction.

### 3 ISOLATION



Concurrent transactions do not interfere with each other. They appear to run in isolation.

#### EXAMPLE

Two users updating the same data at the same time will not see each other's intermediate data.

### 4 DURABILITY



Once a transaction is committed, the changes are permanent even if the system fails.

#### EXAMPLE

Committed changes are stored on disk (redo log) and survive crashes.

### 5 TRANSACTION LIFECYCLE



- **START TRANSACTION:** Begin a new transaction.
- **EXECUTE OPERATIONS:** Perform SQL statements.
- **COMMIT:** Save all changes permanently.
- **ROLLBACK:** Undo all changes if an error occurs.
- **END:** Transaction is closed.

### 6 SAVEPOINTS

- Savepoints allow partial rollback within a transaction.
- Useful for complex transactions with multiple steps.
- Does not commit the transaction.

#### SAVEPOINT EXAMPLE

```

START TRANSACTION;
UPDATE accounts SET balance = balance - 100
WHERE id = 1;
SAVEPOINT after_debit;
UPDATE accounts SET balance = balance + 100
WHERE id = 2;
-- If something goes wrong
ROLLBACK TO SAVEPOINT after_debit;
COMMIT;
  
```

### 7 COMMIT vs ROLLBACK

#### COMMIT



- Makes all changes permanent.
- Data is saved to the database.
- Cannot be undone.

#### ROLLBACK



- Undoes all changes in the transaction.
- Returns the database to the previous state.
- Useful when errors occur.

## COMMON SQL COMMANDS FOR TRANSACTIONS

- |                               |                             |
|-------------------------------|-----------------------------|
| • START TRANSACTION;          | -- Begin a new transaction  |
| • COMMIT;                     | -- Save changes permanently |
| • ROLLBACK;                   | -- Undo all changes         |
| • SAVEPOINT name;             | -- Create a savepoint       |
| • ROLLBACK TO SAVEPOINT name; | -- Rollback to savepoint    |
| • RELEASE SAVEPOINT name;     | -- Remove a savepoint       |
| • SET AUTOCOMMIT = 0;         | -- Disable auto commit      |
| • SET AUTOCOMMIT = 1;         | -- Enable auto commit       |

## BEST PRACTICES

- ✓ Keep transactions short and focused.
- ✓ Avoid long-running transactions.
- ✓ Use appropriate isolation levels.
- ✓ Handle errors and rollback properly.
- ✓ Test transactions under failure conditions.



#### PRO TIP:

Use transactions for ALL critical operations that modify data. They are your first line of defense against data corruption and ensure business reliability.

# CONCURRENCY AND LOCKING

CONTROL CONFLICT. PROTECT DATA. KEEP SYSTEMS RELIABLE.

## KEY TAKEAWAY

- ✓ Locks ensure data integrity in concurrency.
- ✓ Know your lock types.
- ✓ Monitor locks actively.
- ✓ Solve deadlocks fast.
- ✓ Design for high concurrency.

— VERIQTA

## 1 ROW LOCKS



- Lock individual rows.
- Allow other rows to be locked and modified.
- Used by InnoDB for most DML operations.

### EXAMPLE

Two users update different rows in the same table at the same time. Both succeed.

## 2 TABLE LOCKS

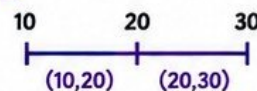


- Lock the entire table.
- No other session can read or write the table.
- Used by MyISAM and for some operations in InnoDB (e.g., ALTER TABLE).

### EXAMPLE

One user locks a table for writing; all other users must wait.

## 3 GAP LOCKS



- Lock gaps between index records.
- Prevents phantom rows.
- Used under REPEATABLE READ isolation level.

### EXAMPLE

Prevent inserting a new row with a key value between 10 and 20.

## 4 NEXT-KEY LOCKING



- Combination of row lock + gap lock.
- Locks the index record and the gap before it.
- Prevents phantoms in range queries.

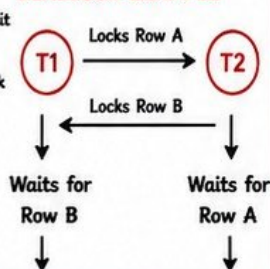
### EXAMPLE

A range query locks both the rows and the gaps it scans.

## 5 DEADLOCKS

- Occur when two or more transactions wait for locks held by each other.
- InnoDB detects deadlocks and rolls back one transaction.
- Keep transactions short and access resources in consistent order.

### DEADLOCK EXAMPLE



Deadlock! One transaction is rolled back.

## 6 LOCK MONITORING



- Monitor locks to identify blocking and performance issues.
- Key tools and commands:

```
SHOW ENGINE INNODB STATUS\G
SELECT * FROM information_schema.innodb_locks;
SELECT * FROM information_schema.innodb_lock_waits;
sys.schema_lock_waits
```

- Watch for long-running transactions holding locks.

## 7 PRODUCTION TROUBLESHOOTING



- Identify blockers and blocked queries.
- Kill or optimize long-running transactions.
- Use low isolation when appropriate.
- Add proper indexes to reduce lock scope.
- Tune InnoDB settings for your workload.

### CHECKLIST

- ☒ Find blocking sessions
- ☒ Review lock waits
- ☒ Analyze queries
- ☒ Optimize or kill blockers
- ☒ Validate after changes

## LOCK TYPES SUMMARY

| LOCK TYPE     | SCOPE       | LOCKS WHAT                | ISOLATION LEVEL         | PURPOSE                      | HIGH CONCURRENCY? |
|---------------|-------------|---------------------------|-------------------------|------------------------------|-------------------|
| ROW LOCK      | Row         | Single index record       | All (InnoDB)            | Protect individual rows      | YES               |
| TABLE LOCK    | Table       | Entire table              | All (depends on engine) | Protect table structure/data | NO                |
| GAP LOCK      | Index Gap   | Gap between index records | REPEATABLE READ         | Prevent phantom rows         | MEDIUM            |
| NEXT-KEY LOCK | Index Range | Record + gap before it    | REPEATABLE READ         | Prevent phantom rows         | MEDIUM            |

## BEST PRACTICES

- ✓ Keep transactions short.
- ✓ Access tables/rows in consistent order.
- ✓ Use the appropriate isolation level.
- ✓ Add indexes to reduce lock range.
- ✓ Monitor locks and resolve issues early.
- ✓ Test concurrency under production load.

## ISOLATION LEVEL IMPACT

| ISOLATION LEVEL | READS               | PHANTOMS  | LOCKING BEHAVIOR     |
|-----------------|---------------------|-----------|----------------------|
| READ COMMITTED  | Latest committed    | Possible  | Minimal locks        |
| REPEATABLE READ | Consistent snapshot | Prevented | Gap + Next-key locks |
| SERIALIZABLE    | Fully consistent    | Prevented | Very strict locking  |

## QUICK TIP



Good index design + short transactions + proper isolation = High concurrency with data integrity.

# REPLICATION ARCHITECTURE

BUILD SCALABLE, HIGH AVAILABLE, AND RESILIENT MYSQL SYSTEMS.

## KEY TAKEAWAY

- ✓ Replication ensures availability and scale.
- ✓ Binary logs power all replication.
- ✓ Use GTID for better reliability.
- ✓ Design for failover and data consistency.

— VERIQTA

## 1 BINARY LOGS

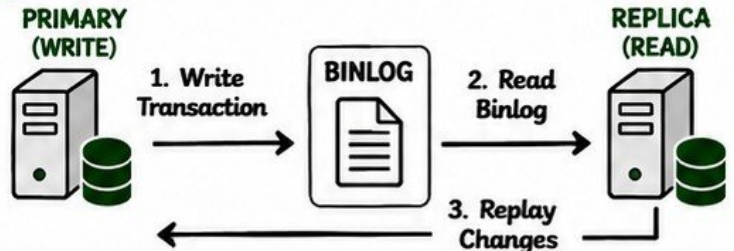


- Binary logs (binlogs) record all changes to the database.
- They are the source of replication.
- Stored on the primary server.
- Includes all DML and DDL changes (based on format).

### BINLOG FORMATS

|           |                                                  |
|-----------|--------------------------------------------------|
| STATEMENT | Logs SQL statements. Smaller size but less safe. |
| ROW       | Logs row changes. Safer and recommended.         |
| MIXED     | Automatically chooses between STATEMENT and ROW. |

## 2 PRIMARY-REPLICA REPLICATION



- ✓ Writes go to the Primary.
- ✓ Changes are recorded in the Binary Log.
- ✓ Replica (Slave) reads and applies the changes.
- ✓ Provides read scaling and high availability.

## 3 GTID (GLOBAL TRANSACTION ID)



- GTID uniquely identifies every transaction.
- Format: <server\_uuid>:<transaction\_id>
- Eliminates the need for file/position.
- Simplifies failover and recovery.
- Prevents duplicate transactions.

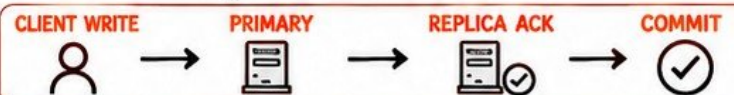
### EXAMPLE GTID

3E11FA47-71CA-11E1-9E33-C80AA9429562:23

## 4 SEMI-SYNCHRONOUS REPLICATION



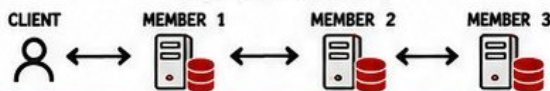
- Primary waits for at least one Replica to acknowledge the transaction.
- Reduces data loss compared to async.
- Better durability with minimal latency impact.
- Configurable WAIT timeout.



## 5 GROUP REPLICATION

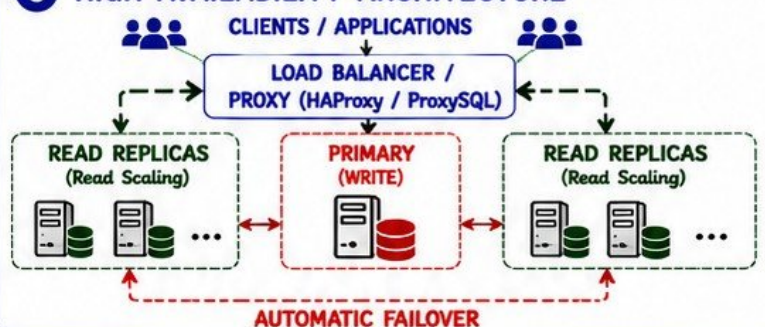


- Multi-primary replication.
- All members can accept writes.
- Built-in conflict detection.
- Automatic member management.
- Provides high availability without single point of failure.



All changes are replicated and certified across the group.

## 6 HIGH AVAILABILITY ARCHITECTURE



### REPLICATION BENEFITS

- ✓ High Availability and Failover
- ✓ Read Scalability
- ✓ Data Redundancy and Safety
- ✓ Disaster Recovery
- ✓ Geographic Distribution



### BEST PRACTICES

- ✓ Use GTID for all new deployments.
- ✓ Enable binary log and set proper retention.
- ✓ Monitor replication lag continuously.
- ✓ Use semi-synchronous replication for critical data.
- ✓ Test failover and recovery regularly.
- ✓ Keep replicas in the same MySQL version.



## PRO TIP:

Design replication with monitoring, alerts, backups, and automated failover. A strong architecture today prevents outages tomorrow!

# BACKUP AND RECOVERY

PROTECT YOUR DATA. RECOVER WITH CONFIDENCE.

## KEY TAKEAWAY

- ✓ Backups are your safety net.
- ✓ Test restores regularly.
- ✓ Plan for disasters, not luck.
- ✓ RPO + RTO define readiness.

— VERIQTA

## 1 MYSQLDUMP



- Logical backup tool provided with MySQL.
- Creates SQL dumps of databases or tables.
- Easy to use and portable.
- Best for small to medium databases.

### EXAMPLE COMMAND

```
mysqldump -u root -p --routines --triggers \
--single-transaction --databases mydb > mydb.sql
```

### PROS

- Simple and easy to use
- Portable (SQL format)
- Good for small datasets

### CONS

- Slow for large databases
- Locks tables (unless options used)
- Not suitable for hot backup of large DBs

## 2 MYSQL ENTERPRISE BACKUP



- Hot backup solution from Oracle.
- Non-blocking, fast, and reliable.
- Incremental backups supported.
- Consistent backups for InnoDB without locking tables.

### EXAMPLE COMMANDS

```
mysqlbackup --backup-dir=/backup/full --user=root --password=***
mysqlbackup --backup-dir=/backup/incremental --incremental
```

### PROS

- Very fast and non-blocking
- Incremental backups
- Enterprise support

### CONS

- Commercial (licensed)
- Less flexible than XtraBackup
- Requires setup and learning

## 3 PERCONA XTRABACKUP



- Open-source hot backup tool for InnoDB.
- Non-blocking and consistent backups.
- Supports full, incremental, and prepared backups.
- Widely used in production environments.

### EXAMPLE COMMAND

```
xtrabackup --backup --target-dir=/backup/full
--user=root --password=***
```

### PROS

- Open source
- Fast and reliable
- Supports compression
- Flexible and powerful

### CONS

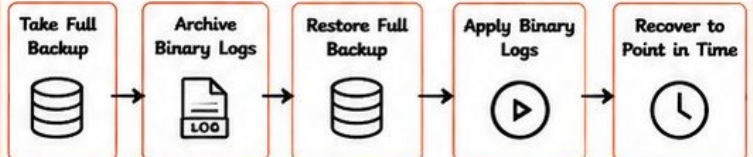
- Steeper learning curve
- More manual steps
- Requires preparation for PITR

## 4 POINT-IN-TIME RECOVERY (PITR)



- Restores the database to a specific time before a failure.
- Requires binary logs and a full backup.
- Essential for recovering from user errors, deletes, or corruption.
- Minimizes data loss (low RPO).

### PITR PROCESS FLOW



## 5 BACKUP VALIDATION



- Always validate backups to ensure they are usable.
- Perform test restores regularly.
- Verify data integrity after restore.
- Automate validation where possible.

### VALIDATION CHECKLIST

- ☒ Restore backup to a test server
- ☒ Check database objects (tables, views, procedures)
- ☒ Verify row counts and important data
- ☒ Check application connectivity
- ☒ Document results and issues

## 6 DISASTER RECOVERY STRATEGY



- Plan for disasters before they happen.
- Define RPO (Recovery Point Objective) and RTO (Recovery Time Objective).
- Keep backups offsite or in another region.
- Test your DR plan regularly.
- Have clear roles and communication plan.

### DR STRATEGY COMPONENTS



### PRO TIP:

A backup is not a backup until it is RESTORED and VERIFIED. Automate backups. Test restores. Document everything.

# PERFORMANCE MONITORING

MEASURE. ANALYZE. OPTIMIZE. DELIVER EXCEPTIONAL PERFORMANCE.

## KEY TAKEAWAY

- ✓ Monitor continuously.
- ✓ Identify bottlenecks early.
- ✓ Use the right metrics.
- ✓ Build meaningful dashboards.
- ✓ Act before users are affected.

— VERIQTA

## 1 PERFORMANCE SCHEMA



- Built-in instrumentation for deep performance insights.
- Captures events, waits, stages, and transactions.
- Low overhead and highly configurable.
- Essential for advanced troubleshooting.

### USEFUL TABLES

|                                           |                            |
|-------------------------------------------|----------------------------|
| events_statements_summary_by_digest       | Top SQL by latency         |
| events_waits_summary_global_by_event_name | Wait events summary        |
| threads                                   | Thread and connection info |
| table_io_waits_summary_by_table           | I/O by table               |
| file_summary_by_instance                  | File I/O statistics        |

## 2 INFORMATION SCHEMA



- Provides metadata about server, databases, tables, and processes.
- Lightweight and always available.
- Great for inventory and basic monitoring.

### COMMONLY USED TABLES

|                  |                         |
|------------------|-------------------------|
| PROCESSLIST      | Current running queries |
| TABLES           | Table information       |
| SCHEMATA         | Database information    |
| GLOBAL STATUS    | Server status variables |
| GLOBAL VARIABLES | Server configuration    |

## 3 SLOW QUERY LOG



- Logs queries that exceed a defined execution time.
- Helps identify expensive and inefficient queries.
- Key tool for optimization.

### EXAMPLE CONFIGURATION

```
slow_query_log = ON
long_query_time = 1
slow_query_log_file = /var/log/mysql/mysql-slow.log
```

## 4 ERROR LOG



- Logs server start/stop, warnings, errors, and critical issues.
- First place to check when something goes wrong.
- Essential for troubleshooting and audits.

### VIEW ERROR LOG

```
SHOW VARIABLES LIKE 'log_error';
tail -f /var/log/mysql/error.log
```

## 5 GENERAL LOG



- Logs all client connections and all statements.
- Very useful for auditing and debugging.
- High overhead – use with care in production.

### ENABLE / DISABLE

```
SET GLOBAL general_log = ON;
SET GLOBAL general_log_file = '/var/log/mysql/mysql-general.log';
SET GLOBAL general_log = OFF;
```

## 6 KEY PERFORMANCE METRICS



- QPS (Queries Per Second): Overall load.
- TPS (Transactions Per Second): Throughput.
- Latency: Response time of queries.
- CPU Usage: Server CPU consumption.
- Memory Usage: Buffer pool and memory use.
- Disk I/O: Read/Write IOPS and latency.
- Connections: Active and max connections.
- Slow Queries: Number and impact.

## 7 PRODUCTION DASHBOARDS



- Visualize metrics in real-time.
- Detect issues before impact.
- Correlate metrics for deep insights.
- Use thresholds and alerts.

### DASHBOARD TOOLS

- MySQL Enterprise Monitor
- Percona Monitoring and Management (PMM)
- Grafana + Prometheus + MySQL Exporter
- Datadog / New Relic / CloudWatch

## 8 BEST PRACTICES



- Monitor 24/7 with alerts.
- Keep slow\_query\_log enabled.
- Review error logs daily.
- Track baseline before tuning.
- Use dashboards for quick visibility.
- Automate monitoring and reporting.



### PRO TIP:

You can't improve what you don't measure.  
Monitor everything, focus on what matters, and act fast!

# PERFORMANCE TUNING

FINE-TUNE MYSQL FOR MAXIMUM SPEED AND EFFICIENCY.

## KEY TAKEAWAY

- ✓ Tune systematically.
- ✓ Measure before and after changes.
- ✓ Understand your workload.
- ✓ Small improvements compound.

— VERIQTA

## 1 BUFFER POOL SIZING



- The Buffer Pool caches data and indexes in memory.
- Larger Buffer Pool = more data in memory, fewer disk reads.
- Ideal size is 70–80% of total RAM (dedicated MySQL server).
- Monitor hit ratio to validate.

### CONFIG EXAMPLE

`innodb_buffer_pool_size = 70–80% of RAM`  
(Check with `SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool%';`)

### KEY METRIC

`Innodb_buffer_pool_reads / Innodb_buffer_pool_read_requests`  
Lower is better (target < 1%).

## 3 CONNECTION TUNING



- Too many connections = memory usage and context switching.
- Tune based on workload and server capacity.
- Use connection pooling (e.g., ProxySQL, PgBouncer, or application pooling).
- Monitor `Threads_connected`.

### IMPORTANT VARIABLES

`max_connections` : Maximum allowed connections  
`thread_cache_size` : Reuse thread structures  
`wait_timeout` : Close idle non-interactive connections  
`interactive_timeout` : Close idle interactive connections

## 5 DISK I/O OPTIMIZATION



- Disk I/O is the biggest bottleneck.
- Use SSD or NVMe storage.
- Place data files, logs, and temp files on fast storage.
- Separate I/O heavy files across different disks if possible.

### BEST PRACTICES

- ✓ Use RAID 10 for high IOPS and redundancy.
- ✓ Use `noatime` mount option.
- ✓ Place redo logs and temporary tables on fast disks.
- ✓ Monitor with `iostat`, `iotop`, and Performance Schema.

## 2 QUERY CACHE CONSIDERATIONS



- Query Cache can speed up identical `SELECT` queries.
- In write-heavy workloads, it can cause contention.
- MySQL 8.0 removed Query Cache.
- Focus on other cache layers instead.

### RECOMMENDATION

Disable in write-heavy systems:  
`query_cache_type = 0`  
`query_cache_size = 0`

### ALTERNATIVES

Use application cache (Redis/Memcached), proper indexing, and Buffer Pool.

## 4 TEMPORARY TABLES



- Created for `ORDER BY`, `GROUP BY`, `DISTINCT`, `UNION`, etc.
- If too large, they spill to disk (slows performance).
- Keep temporary tables in memory when possible.

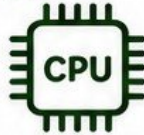
### KEY VARIABLES

`tmp_table_size` : Max size of in-memory temp table  
`max_heap_table_size` : Max size of MEMORY temp table  
Set both to the same value.

### BEST PRACTICE

Size based on workload and available memory.

## 6 CPU OPTIMIZATION



- High CPU usage slows query execution.
- Optimize slow queries and remove inefficient scans.
- Use proper indexing to reduce CPU cost.
- Avoid complex joins and functions on large datasets.

### BEST PRACTICES

- ✓ Analyze and optimize queries regularly.
- ✓ Use `EXPLAIN` to inspect execution plans.
- ✓ Avoid unnecessary full table scans.
- ✓ Keep statistics updated (`ANALYZE TABLE`).

## 7 MEMORY TUNING



- MySQL uses memory for caches, buffers, threads, and sorting.
- Too little = more disk I/O. Too much = OS swapping (very bad).
- Balance all memory consumers.

### KEY VARIABLES

`innodb_buffer_pool_size` : Main memory consumer  
`sort_buffer_size` : Per-thread sorting buffer  
`join_buffer_size` : Per-thread join buffer  
`read_buffer_size` : Per-thread read buffer  
`read_rnd_buffer_size` : Per-thread random read buffer  
`innodb_log_buffer_size` : InnoDB log buffer

⚡ Monitor total memory usage. Leave enough for OS and other processes.



### PRO TIP:

Change one thing at a time. Test under real workload. Measure, validate, and document every tuning change.

# SECURITY BEST PRACTICES

SECURE YOUR MYSQL. PROTECT YOUR DATA. STAY COMPLIANT.

## KEY TAKEAWAY

- ✓ Secure users and access.
- ✓ Encrypt data in transit and at rest.
- ✓ Audit and monitor continuously.
- ✓ Follow least privilege principle.

— VERIQTA

## 1 USER MANAGEMENT



- Create dedicated users for applications, services, and DBAs.
- Avoid using the root account for daily operations.
- Remove inactive or unused users.
- Use strong authentication methods.

### BEST PRACTICES

- ✓ Use meaningful usernames.
- ✓ Limit user access to specific hosts.
- ✓ Review and clean up users regularly.
- ✓ Disable anonymous users.

## 2 ROLES AND PRIVILEGES



- Apply the principle of least privilege.
- Grant only the privileges required for the task.
- Use roles (MySQL 8.0+) to group privileges.
- Revoke unnecessary privileges regularly.

### BEST PRACTICES

- ✓ Use GRANT and REVOKE carefully.
- ✓ Prefer roles over direct privilege grants.
- ✓ Avoid granting global privileges.
- ✓ Audit privileges periodically.

## 3 SSL/TLS ENCRYPTION



- Encrypt data in transit between clients and the server.
- Use TLS 1.2 or higher.
- Enforce encrypted connections for all clients.
- Validate server certificates.

### BEST PRACTICES

- ✓ Enable REQUIRE SSL or REQUIRE X509.
- ✓ Use strong cipher suites.
- ✓ Keep certificates up to date.
- ✓ Test connections with SSL regularly.

## 4 DATA ENCRYPTION



- Encrypt sensitive data at rest.
- Use Transparent Data Encryption (TDE) for tablespaces and logs.
- Encrypt backup files.
- Protect key management securely.

### BEST PRACTICES

- ✓ Enable MySQL Enterprise TDE (if available).
- ✓ Encrypt backups and binary logs.
- ✓ Secure encryption keys and limit access.
- ✓ Regularly verify encryption is enabled.

## 5 PASSWORD POLICIES



- Enforce strong password policies.
- Use strong passwords with mixed characters.
- Set password expiration.
- Prevent password reuse.

### BEST PRACTICES

- ✓ Use validate\_password plugin.
- ✓ Enforce minimum length, complexity, and history.
- ✓ Set password\_expiration.
- ✓ Avoid shared or default passwords.

## 6 AUDIT LOGGING



- Enable audit logging to track user activity.
- Log authentication, privileges changes, and critical queries.
- Store logs securely and monitor regularly.
- Integrate with SIEM solutions.

### BEST PRACTICES

- ✓ Enable MySQL Enterprise Audit (or community plugins).
- ✓ Log failed logins and privilege changes.
- ✓ Protect audit logs from tampering.
- ✓ Monitor logs and set up alerts.

## 7 COMPLIANCE CONSIDERATIONS



- Align with standards: GDPR, HIPAA, PCI DSS, SOX, ISO 27001.
- Classify and protect sensitive data.
- Maintain access controls and audit trails.
- Perform regular security assessments.
- Document security policies and procedures.

### BEST PRACTICES

- ✓ Map controls to compliance requirements.
- ✓ Keep detailed records for audits.
- ✓ Work closely with security and compliance teams.



### PRO TIP:

Security is not a feature. It is a process.  
Apply least privilege, encrypt everything, audit always!

# PARTITIONING AND LARGE DATABASES

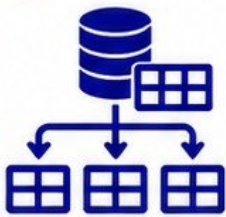
DESIGN FOR SCALE. MANAGE GROWTH. IMPROVE PERFORMANCE.

## KEY TAKEAWAY

- ✓ Partition to improve performance and manage data growth.
- ✓ Keep queries fast and systems scalable.
- ✓ Plan archiving and retention early.

— VERIQTA

## 1 HORIZONTAL PARTITIONING

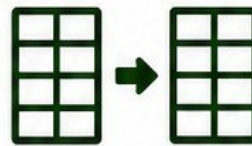


- Split rows across partitions based on a key (e.g., date, region, customer\_id).
- Each partition contains a subset of rows.
- Reduces I/O and improves query performance.

### BEST PRACTICES

- ✓ Partition on columns used in WHERE clauses.
- ✓ Use RANGE or LIST partitioning for time-based data.
- ✓ Keep partition size balanced.
- ✓ Monitor and maintain partitions regularly.

## 2 VERTICAL PARTITIONING



- Split columns into separate tables.
- Keep frequently accessed columns in one table.
- Move rarely accessed columns to another table.

### BEST PRACTICES

- ✓ Separate large BLOB/TEXT columns.
- ✓ Normalize and index properly.
- ✓ Use JOINS carefully to avoid performance hits.
- ✓ Review access patterns before splitting.

## 3 PARTITION PRUNING



- MySQL skips partitions that do not match the query condition.
- Reduces the amount of data scanned.
- Essential for performance with large datasets.

### BEST PRACTICES

- ✓ Always include partition key in queries.
- ✓ Use sargable conditions (avoid functions on keys).
- ✓ Verify pruning using EXPLAIN PARTITIONS.
- ✓ Test and validate query performance.

## 4 LARGE TABLE MANAGEMENT



- Large tables slow down queries, backups, and maintenance.
- Partitioning, indexing, and proper design help manage size.
- Monitor growth and act early.

### BEST PRACTICES

- ✓ Use proper indexing and avoid unnecessary indexes.
- ✓ Regularly analyze and optimize tables.
- ✓ Monitor table size and growth rate.
- ✓ Use tools like pt-online-schema-change for changes.

## 5 ARCHIVING STRATEGIES



- Move old or unused data to archive tables or storage.
- Keep production tables small for better performance.
- Ensure archived data remains accessible when needed.

### BEST PRACTICES

- ✓ Define retention and archiving policies.
- ✓ Use partitioning to drop old partitions.
- ✓ Archive to separate databases or storage.
- ✓ Automate archiving and cleanup jobs.

## 6 SCALING CONSIDERATIONS



- Partitioning helps scale read and write operations.
- Consider sharding for extreme scale.
- Use read replicas to offload read traffic.
- Plan for growth from early stages.

### BEST PRACTICES

- ✓ Monitor metrics: QPS, latency, disk, CPU.
- ✓ Use sharding only when necessary.
- ✓ Combine partitioning with replication.
- ✓ Continuously review and adjust architecture.



### PRO TIP:

A good partitioning strategy improves performance today and reduces operational pain tomorrow.

# AUTOMATION AND ADMINISTRATION

AUTOMATE TASKS. ENSURE HEALTH. OPERATE WITH CONFIDENCE.

## KEY TAKEAWAY

- ✓ Automate routine tasks.
- ✓ Maintain database health.
- ✓ Reduce manual errors.
- ✓ Standardize operations.
- ✓ Improve reliability and efficiency.

— VERIQTA

## 1 SCHEDULED MAINTENANCE



- Schedule routine tasks like backups, index rebuilds, and statistics updates.
- Use OFF-PEAK hours to minimize impact.
- Keep maintenance windows consistent and documented.

### BEST PRACTICES

- ✓ Define maintenance windows clearly.
- ✓ Automate with Event Scheduler or cron.
- ✓ Test maintenance tasks before production.
- ✓ Monitor task duration and resource usage.

## 2 EVENT SCHEDULER



- MySQL Event Scheduler runs tasks at specified times.
- Automate backups, purges, and report generation.
- Supports one-time and recurring tasks.

### BEST PRACTICES

- ✓ Enable the Event Scheduler (event\_scheduler=ON).
- ✓ Store events in a dedicated schema.
- ✓ Monitor event execution and failures.
- ✓ Keep events lightweight and idempotent.

## 3 HEALTH CHECKS



- Continuously check database availability and performance.
- Monitor connections, replication, disk space, and slow queries.
- Alert early to prevent outages.

### BEST PRACTICES

- ✓ Monitor key metrics: CPU, RAM, Disk, Connections.
- ✓ Check replication lag and binlog status.
- ✓ Validate backups and disk space regularly.
- ✓ Configure alerts for critical thresholds.

## 4 DATABASE AUTOMATION



- Automate provisioning, backups, failover, and user management.
- Use scripts and tools to ensure consistency and repeatability.
- Reduce manual work and human errors.

### BEST PRACTICES

- ✓ Use Infrastructure as Code (IaC) where possible.
- ✓ Standardize configurations and templates.
- ✓ Automate backups and restore verification.
- ✓ Log all automated operations.

## 5 PYTHON AUTOMATION



- Use Python (mysql-connector) to automate database tasks.
- Build scripts for backups, monitoring, and reporting.
- Integrate with APIs, alerts, and dashboards.

### BEST PRACTICES

- ✓ Use secure credential storage.
- ✓ Handle exceptions and log errors.
- ✓ Modularize and reuse code.
- ✓ Schedule scripts with cron or Event Scheduler.

## 6 OPERATIONAL CHECKLISTS



- Use checklists to ensure consistency in operations.
- Document procedures for daily, weekly, and monthly tasks.
- Review and update regularly.

### BEST PRACTICES

- ✓ Create role-based operational checklists.
- ✓ Include verification and rollback steps.
- ✓ Store checklists in version-controlled repo.
- ✓ Perform regular drills and reviews.



## PRO TIP:

AUTOMATION IS NOT JUST ABOUT SAVING TIME. IT IS ABOUT BUILDING RELIABILITY, CONSISTENCY, AND TRUST IN YOUR DATABASE OPERATIONS.

# TROUBLESHOOTING PRODUCTION INCIDENTS

DETECT. DIAGNOSE. RESOLVE. PREVENT RECURRENCE.

## KEY TAKEAWAY

- ✓ Identify the problem quickly.
- ✓ Use the right tools.
- ✓ Follow a structured process.
- ✓ Fix the root cause.
- ✓ Prevent it from happening again.

— VERIQTA

### 1 SLOW DATABASE RESPONSE



- Check slow query log.
- Identify long-running queries.
- Analyze execution plans.
- Review indexes and missing indexes.
- Check for resource saturation.

#### TROUBLESHOOTING STEPS

- ✓ Enable slow query log.
- ✓ Run EXPLAIN on slow queries.
- ✓ Optimize queries or add indexes.
- ✓ Verify buffer pool and disk I/O.
- ✓ Monitor after changes.

### 2 REPLICATION LAG

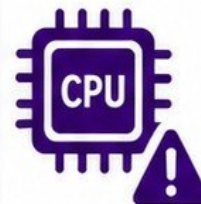


- Check replication status.
- Identify lag on replicas.
- Examine slow queries on replica.
- Check network latency.
- Review I/O and resource constraints.

#### TROUBLESHOOTING STEPS

- ✓ SHOW SLAVE STATUS\G (or SHOW REPLICA STATUS\G).
- ✓ Identify Seconds\_Behind\_Master.
- ✓ Check relay log size.
- ✓ Review long transactions.
- ✓ Optimize queries and hardware if needed.

### 3 HIGH CPU USAGE



- Identify CPU intensive queries.
- Check for full table scans.
- Look for excessive connections.
- Review configuration and background processes.
- Check OS level CPU usage.

#### TROUBLESHOOTING STEPS

- ✓ Use top, htop, or vmstat.
- ✓ Check PROCESSLIST for running queries.
- ✓ Review slow query log.
- ✓ Optimize queries and indexes.
- ✓ Tune MySQL configuration.

### 4 LOCK CONTENTION



- Identify blocked queries.
- Check locks and waiting transactions.
- Review transaction isolation level.
- Shorten long transactions.
- Optimize indexing.

#### TROUBLESHOOTING STEPS

- ✓ SHOW PROCESSLIST; find "Waiting for table lock".
- ✓ Use PERFORMANCE\_SCHEMA.data\_locks.
- ✓ Identify blocking session.
- ✓ Kill or optimize blocking transaction.
- ✓ Improve indexes and queries.

### 5 DISK-FULL INCIDENTS



- Check disk usage.
- Identify large tables and logs.
- Clean up unnecessary files.
- Enable automated cleanup processes.
- Expand disk if needed.

#### TROUBLESHOOTING STEPS

- ✓ df -h to check disk usage.
- ✓ Identify large files (logs, tables, backups).
- ✓ Purge old logs and data.
- ✓ Enable log rotation and archiving.
- ✓ Add disk or increase storage.

### 6 CORRUPTION DETECTION



- Run consistency checks.
- Detect and isolate corruption.
- Restore from backup or replica.
- Prevent with regular backups.
- Monitor storage health.

#### TROUBLESHOOTING STEPS

- ✓ CHECK TABLE ... FOR UPGRADE / CHECKSUM.
- ✓ Use mysqlcheck or MySQL Shell util.checkForErrors().
- ✓ Isolate affected objects.
- ✓ Restore from backup or replica.
- ✓ Investigate root cause (disk, memory, bugs).

### 7 ROOT CAUSE ANALYSIS WORKFLOW



**1. DETECT**  
Alert or user reports issue.

**2. IDENTIFY**  
Confirm problem and scope.

**3. DIAGNOSE**  
Collect data, logs, metrics, queries.

**4. RESOLVE**  
Fix the issue and restore service.

**5. PREVENT**  
Document, monitor, and improve.

#### BEST PRACTICES

- ✓ Monitor continuously.
- ✓ Keep logs and backups.
- ✓ Automate alerts.
- ✓ Document incidents.
- ✓ Learn and improve.

— VERIQTA

# HIGH AVAILABILITY AND DISASTER RECOVERY

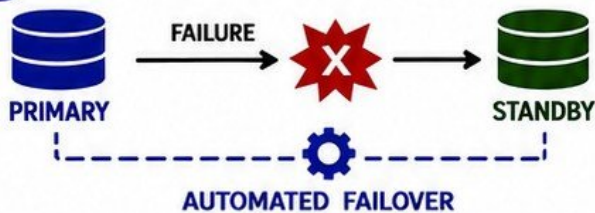
BUILD RESILIENT DATABASES. STAY AVAILABLE.  
RECOVER FAST. PROTECT YOUR DATA.

## KEY TAKEAWAY

- ✓ Design for failure.
- ✓ Replicate your data.
- ✓ Automate failover.
- ✓ Test recovery regularly.
- ✓ Know your RTO and RPO.

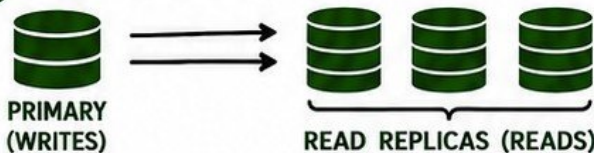
— VERIQTA

## 1 FAILOVER STRATEGIES



- Automatically switch to a standby server when the primary fails.
- Reduce downtime and maintain application availability.
- Use tools like Orchestrator, MHA, or ProxySQL for automation.
- Regularly test failover to ensure it works when needed.

## 2 READ REPLICAS



- Offload read traffic to replicas.
- Improve performance and scalability.
- Replicas are asynchronous by default.
- Monitor replication lag continuously.
- Use Semi-Sync Replication for higher consistency when needed.

## 3 MULTI-REGION DEPLOYMENTS



- Deploy across multiple regions for high availability and disaster recovery.
- Protect against region-wide failures.
- Use global load balancing and DNS routing policies.
- Consider latency, data residency, and cost.

## 4 PROXYSQL



PROXYSQL

- ✓ Query routing
- ✓ Connection pooling
- ✓ Load balancing
- ✓ Failover automation
- ✓ Read/Write splitting

- High-performance MySQL proxy.
- Intelligent query routing and connection pooling.
- Automatic failover and host monitoring.
- Read/Write splitting and replication awareness.
- Improves scalability and availability.

## 5 LOAD BALANCING



- Distribute traffic across healthy nodes.
- Improve performance and availability.
- Use health checks to remove unhealthy nodes.
- Support both read and write traffic.
- Tools: ProxySQL, HAProxy, AWS NLB/ALB.

## 6 RECOVERY OBJECTIVES (RTO / RPO)

**RTO (RECOVERY TIME OBJECTIVE)**  
The maximum acceptable time to restore the database and resume operations.  
How fast do you need to recover?

**RPO (RECOVERY POINT OBJECTIVE)**  
The maximum acceptable amount of data loss measured in time.  
How much data can you afford to lose?

- Define RTO and RPO based on business needs.
- Design architecture to meet these objectives.
- Use backups, replicas, and automation.
- Test recovery plans regularly.
- Document procedures and responsibilities.



## BEST PRACTICES

- ✓ Test failover and backups regularly.
- ✓ Monitor replication lag, disk space, and system health.
- ✓ Automate recovery and document runbooks.
- ✓ Design for redundancy, not just performance.



# PRODUCTION CASE STUDIES

REAL PROBLEMS. REAL INVESTIGATIONS. REAL SOLUTIONS.

## KEY TAKEAWAY

- ✓ Investigate systematically.
- ✓ Understand the root cause.
- ✓ Apply the right fix.
- ✓ Document and improve.
- ✓ Share lessons learned.

— VERIQTA

## 1 SLOW QUERY INVESTIGATION



### WHAT HAPPENED:

Application slow under load. Users experienced timeouts.

### ROOT CAUSE:

Missing index and inefficient JOIN on a large table.

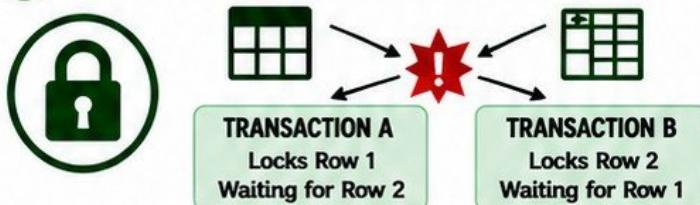
### SOLUTION:

Added composite index, rewrote query, and updated statistics.

### LESSON LEARNED:

Always analyze execution plans and add the right indexes.

## 2 DEADLOCK ANALYSIS



### WHAT HAPPENED:

Transactions failing with deadlock errors.

### ROOT CAUSE:

Transactions locking resources in different order.

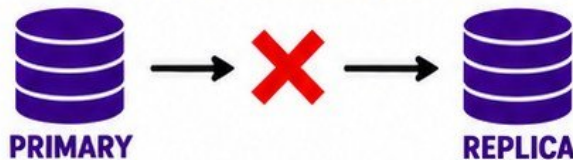
### SOLUTION:

Standardized locking order and reduced transaction scope.

### LESSON LEARNED:

Keep transactions short and lock resources in a consistent order.

## 3 REPLICATION FAILURE RECOVERY



### WHAT HAPPENED:

Replica stopped applying binary logs.

### ROOT CAUSE:

Network issue caused replication lag and a break.

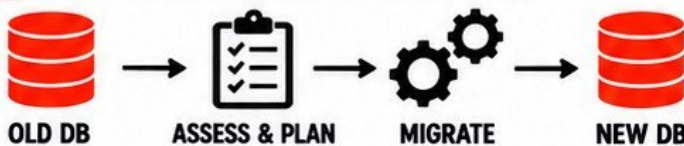
### SOLUTION:

Repositioned replica using GTID and re-synchronized.

### LESSON LEARNED:

Monitor replication lag and test failover regularly.

## 4 DATABASE MIGRATION



### WHAT HAPPENED:

Business required migration to MySQL 8.0.

### ROOT CAUSE:

Legacy features and incompatible SQL modes.

### SOLUTION:

Tested, refactored queries, migrated in phases with validation.

### LESSON LEARNED:

Plan thoroughly and test in a staging environment.

## 5 SCALING A HIGH-TRAFFIC APPLICATION



### WHAT HAPPENED:

Traffic increased 10x during a promotion.

### ROOT CAUSE:

Single database instance became a bottleneck.

### SOLUTION:

Implemented read replicas, ProxySQL, caching, and connection pooling.

### LESSON LEARNED:

Scale horizontally and monitor everything.

## 6 LESSONS LEARNED



- ✓ Monitor proactively, not reactively.
- ✓ Understand your workload and data patterns.
- ✓ Test recovery and failover before incidents.
- ✓ Document runbooks and share knowledge.
- ✓ Review and improve after every incident.

“ Every incident is an opportunity to build a more reliable system.

— VERIQTA

# PRODUCTION READINESS REVIEW

BUILD RELIABLE. SECURE. PERFORMANT. RESILIENT DATABASES.  
USE THIS CHECKLIST BEFORE GOING LIVE.

## KEY TAKEAWAY

- ✓ Prepare thoroughly.
- ✓ Test everything.
- ✓ Automate and monitor.
- ✓ Document and review.
- ✓ Operate with excellence.

— VERIQTA

## 1 DATABASE ARCHITECTURE CHECKLIST



- ✓ Architecture aligns with business goals
- ✓ Proper database sizing and allocation
- ✓ Indexes designed for workload
- ✓ Normalization and data modeling reviewed
- ✓ Partitioning strategy (if needed)
- ✓ Connection pooling configured
- ✓ Read/Write separation (if needed)
- ✓ Scalability reviewed and validated
- ✓ Documentation is complete and accurate

## 2 SECURITY CHECKLIST



- ✓ Strong authentication and least privilege
- ✓ Roles and permissions reviewed
- ✓ Encryption in transit (TLS) enabled
- ✓ Encryption at rest enabled
- ✓ Sensitive data masked or encrypted
- ✓ Audit logging enabled and reviewed
- ✓ Vulnerability scanning completed
- ✓ Database firewall rules configured
- ✓ Regular security reviews scheduled

## 3 PERFORMANCE CHECKLIST



- ✓ Slow query log enabled and reviewed
- ✓ Critical queries optimized and indexed
- ✓ Execution plans analyzed
- ✓ Buffer pool / cache properly sized
- ✓ Temp space and redo logs sized correctly
- ✓ Connection limits configured
- ✓ Database parameters tuned
- ✓ Load testing completed
- ✓ Performance baseline established

## 4 BACKUP AND RECOVERY CHECKLIST



- ✓ Automated backups configured
- ✓ Backup retention policy defined
- ✓ Backup integrity regularly tested
- ✓ Point-in-time recovery enabled
- ✓ Restore process documented and tested
- ✓ Offsite / cross-region backups configured
- ✓ Recovery runbooks are available
- ✓ Backup monitoring and alerting enabled
- ✓ RPO and RTO are defined and met

## 5 HIGH AVAILABILITY CHECKLIST



- ✓ HA architecture implemented
- ✓ Replication topology tested
- ✓ Failover process tested and documented
- ✓ Read replicas monitored
- ✓ Load balancer / Proxy configured
- ✓ Automatic failover validated
- ✓ Multi-AZ / Multi-region strategy in place
- ✓ RPO and RTO meet business requirements
- ✓ HA monitoring and alerting enabled

## 6 MONITORING CHECKLIST



- ✓ Key metrics are monitored
- ✓ Alerts configured for critical thresholds
- ✓ Slow queries are monitored
- ✓ Replication lag is monitored
- ✓ Disk space and inode usage monitored
- ✓ Connection usage is monitored
- ✓ Logs centralized and retained
- ✓ Dashboards created and reviewed
- ✓ On-call runbooks are available

## 7 SENIOR DBA RECOMMENDATIONS



- ✓ Automate everything you can – manual steps cause failures.
- ✓ Test failure scenarios regularly – not just during incidents.
- ✓ Keep documentation updated – future you will thank you.
- ✓ Monitor proactively – issues are easier to prevent than to fix.
- ✓ Understand your workload – data, queries, and growth patterns.
- ✓ Review and improve continuously – excellence is a habit.

## 8 ENTERPRISE PRODUCTION BEST PRACTICES



- ✓ Follow change management for all database changes.
- ✓ Use Infrastructure as Code for repeatable deployments.
- ✓ Keep environments consistent: Dev, Test, Staging, Production.
- ✓ Enforce naming standards and tagging.
- ✓ Review capacity regularly and plan for growth.
- ✓ Build a culture of reliability, observability, and continuous improvement.